Insurance Premium Notifier

Project Report | Python, Flask and SQLite

1. Introduction

Insurance Premium Notifier is a web-based academic application for maintaining insurance policy details, calculating premium due dates, monitoring grace periods and displaying reminders and lapsed-policy alerts.

2. Objectives

Register and manage policies; support monthly and yearly schedules; calculate dates correctly across month ends and leap years; track grace periods; exclude Paid-Up policies; and persist information in SQLite.

3. Technology Stack

Python 3, Flask, SQLite, HTML5, CSS3, Jinja2 templates and Python unittest.

4. System Architecture

The browser sends requests to Flask routes in app.py. The route layer uses services.py for date/status rules and database.py for SQLite access. Jinja2 templates render the dashboard and policy pages.

5. Functional Modules

Policy management: create, edit, list and delete policies, record payment and mark Paid-Up. Date calculation: monthly rollover clamps to the target month's last valid day; yearly rollover handles February 29. Monitoring: computes Active, Due Soon, Grace Period and Lapsed states. Notifications: stores one message per policy and due-cycle event.

6. Database Design

policies: id, unique policy_number, customer_name, customer_email, customer_phone, premium_amount, frequency, start_date, last_payment_date, next_due_date, grace_period, status and paid_up. notifications: id, policy_id foreign key, message, kind and created_at.

7. Status Algorithm

Paid-Up policies are skipped. A due date more than seven days away is Active. A due date from today through seven days away is Due Soon. After the due date and through the grace deadline is Grace Period. After the grace deadline the policy is Lapsed. Notifications are unique for each policy and event type/due cycle.

8. Testing

The automated suite contains 11 unit tests for month-end calculations, leap-year calculations, Active/Due Soon/Grace Period/Lapsed boundaries, zero grace and invalid frequency. Run: python -m unittest discover -s tests -v. A separate browser smoke test could not be completed in this build environment because Flask is not installed here; install requirements.txt and run the application locally for final UI verification.

9. Installation

Create a virtual environment, activate it, install requirements.txt, run python app.py, then visit http://127.0.0.1:5000. See README.md for Windows and macOS/Linux commands.

10. Limitations and Future Work

This academic version displays notifications within the website; it does not send real SMS or email and is not connected to an insurer. Future improvements include login/authentication, role permissions, email/SMS gateway integration, exports, audit logs and production deployment hardening.

11. Conclusion

The project demonstrates policy CRUD, premium schedule calculations, grace-period monitoring, reminder generation and persistent storage in a modular Flask and SQLite application.